

Razor Pages Application

Technology: ASP.NET Core (.NET 7) Razor Pages

Name: Tushar Dev Singh

Objective: The objective of this assignment is to build a Razor Pages application that demonstrates the use of Razor syntax, PageModel classes, and dynamic data handling. The application allows users to add and view items dynamically.

Features Implemented:

The application includes:

- Razor Pages structure
- PageModel classes
- Form handling
- Data binding
- Dynamic list update
- Navigation between pages

Two pages were created:

1. Index page – displays items
2. AddItem page – allows adding new items

Program.cs:

```
var builder = WebApplication.CreateBuilder(args);

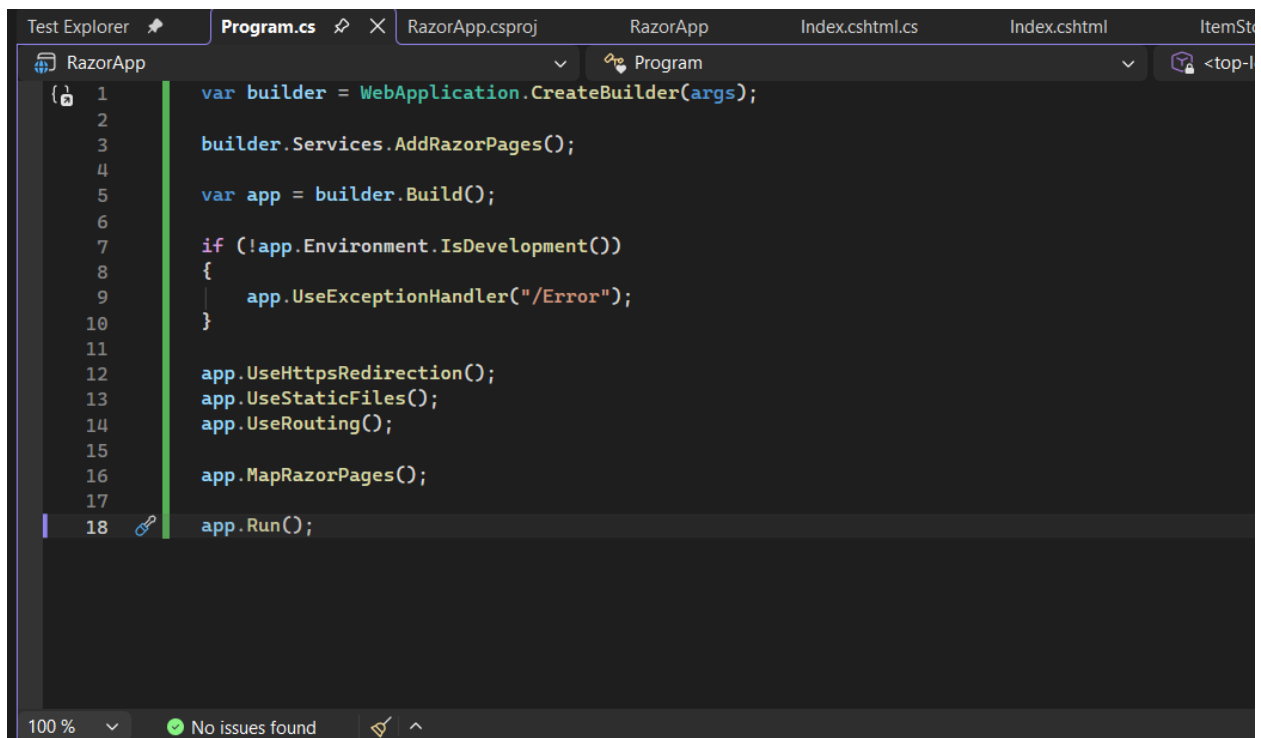
builder.Services.AddRazorPages();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.MapRazorPages();

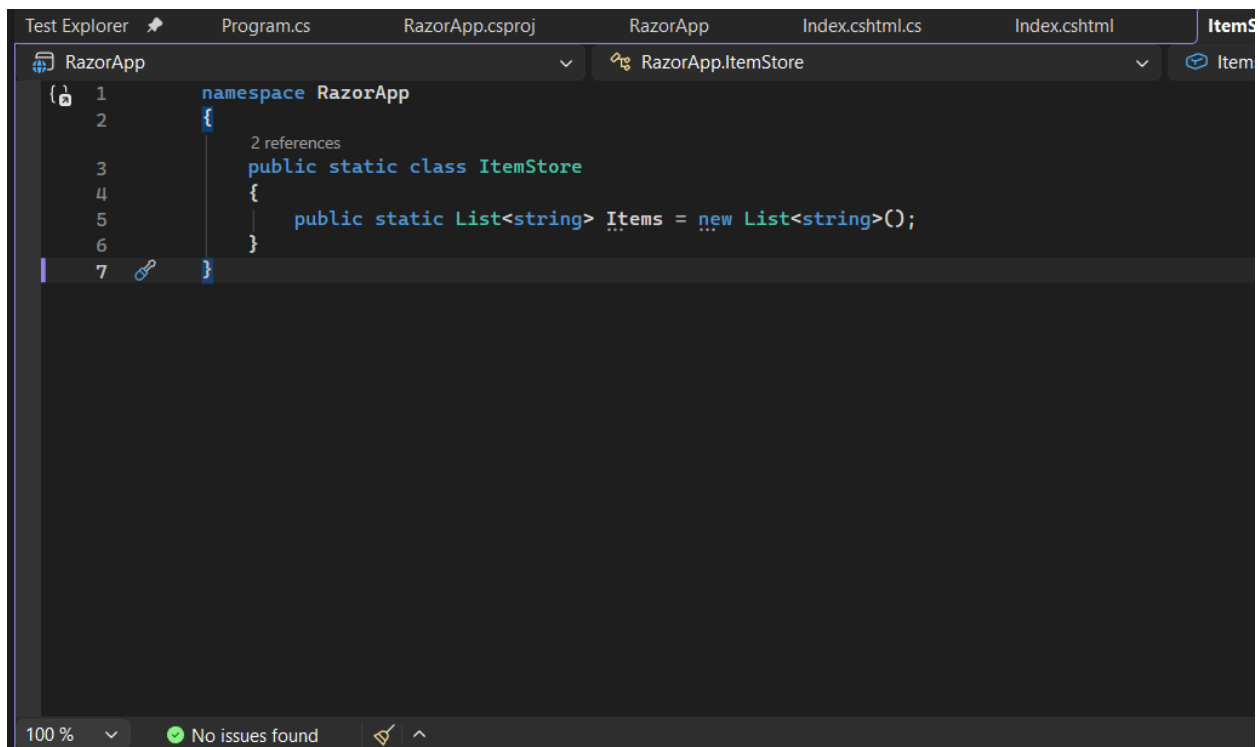
app.Run();
```



Data Storage Class

ItemStore.cs:

```
namespace RazorApp
{
    public static class ItemStore
    {
        public static List<string> Items = new List<string>();
    }
}
```



Index Page (Display Items)

Index.cshtml:

```
@page
@model IndexModel

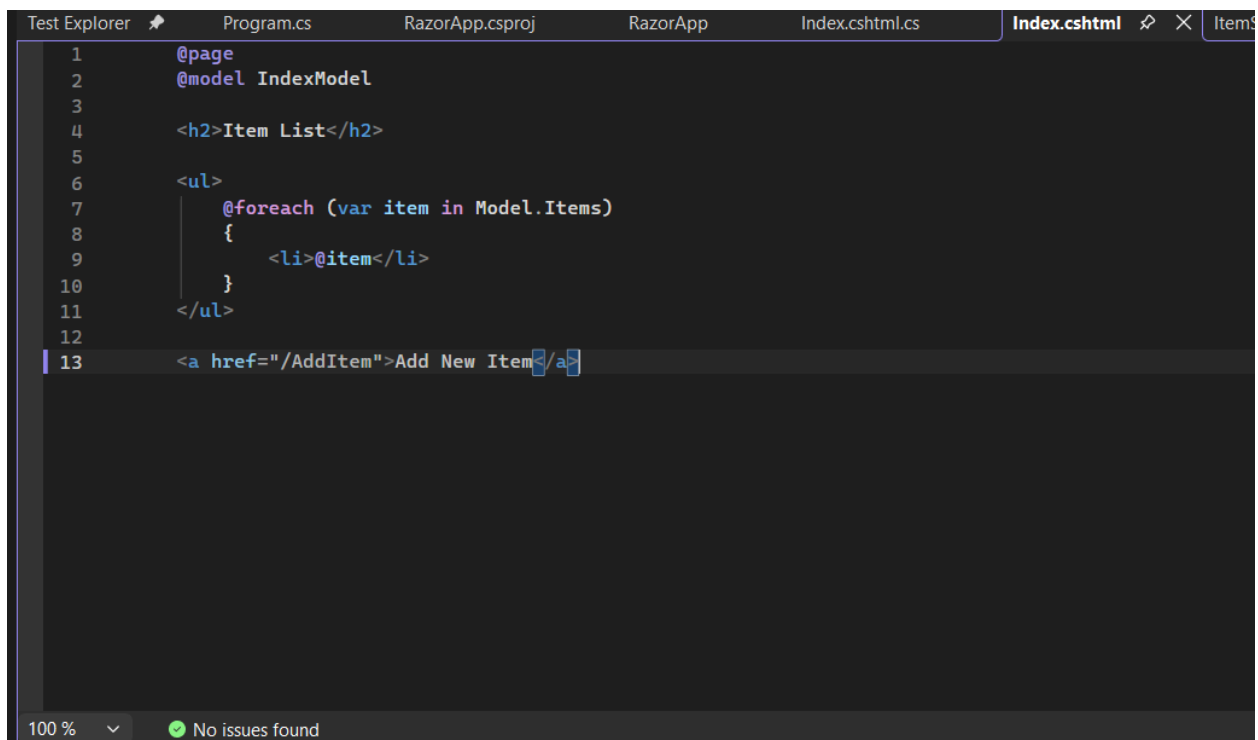
<h2>Item List</h2>

<ul>

    @foreach (var item in Model.Items)
    {
        <li>@item</li>
    }

</ul>

<a href="/AddItem">Add New Item</a>
```



The screenshot shows the Visual Studio Code editor with the 'Index.cshtml' file open. The code is as follows:

```
1  @page
2  @model IndexModel
3
4  <h2>Item List</h2>
5
6  <ul>
7      @foreach (var item in Model.Items)
8      {
9          <li>@item</li>
10     }
11 </ul>
12
13 <a href="/AddItem">Add New Item</a>
```

The interface includes a Test Explorer on the left, a file explorer showing 'Program.cs', 'RazorApp.csproj', 'RazorApp', 'Index.cshtml.cs', and 'Index.cshtml'. The status bar at the bottom indicates '100 %' zoom and 'No issues found'.

Index.cshtml.cs:

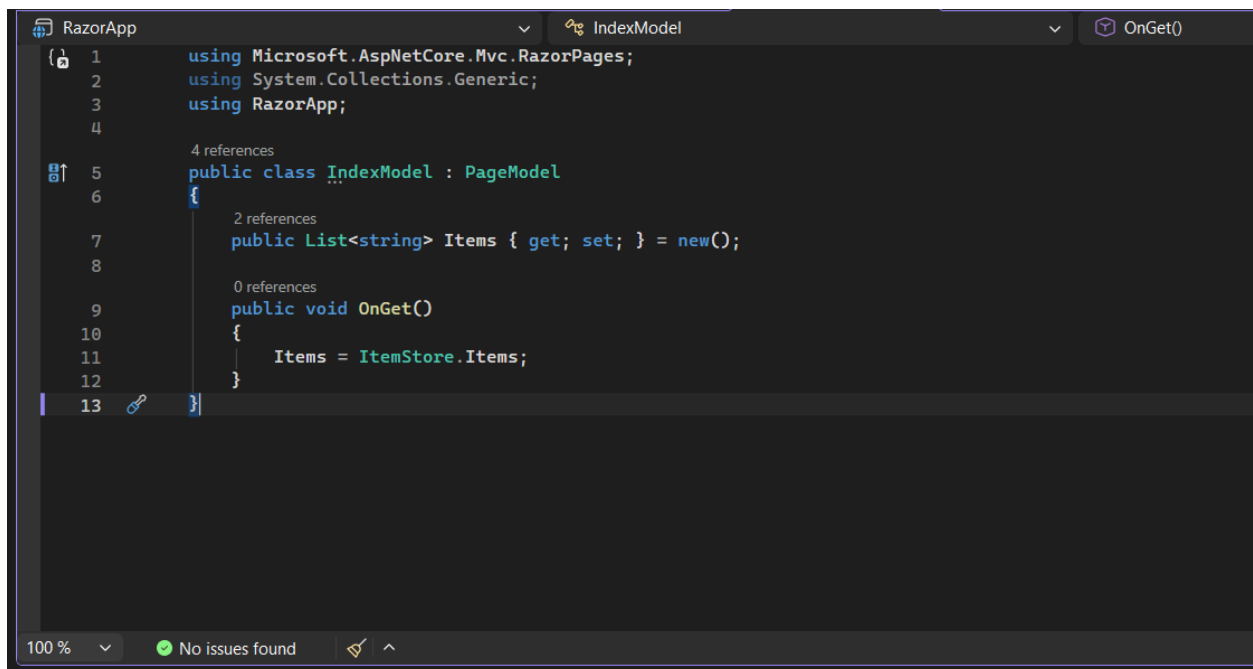
```
using Microsoft.AspNetCore.Mvc.RazorPages;

using System.Collections.Generic;

using RazorApp;

public class IndexModel : PageModel
{
    public List<string> Items { get; set; } = new();

    public void OnGet()
    {
        Items = ItemStore.Items;
    }
}
```



Add Item Page

AddItem.cshtml:

```
@page
@model AddItemModel

<h2>Add Item</h2>

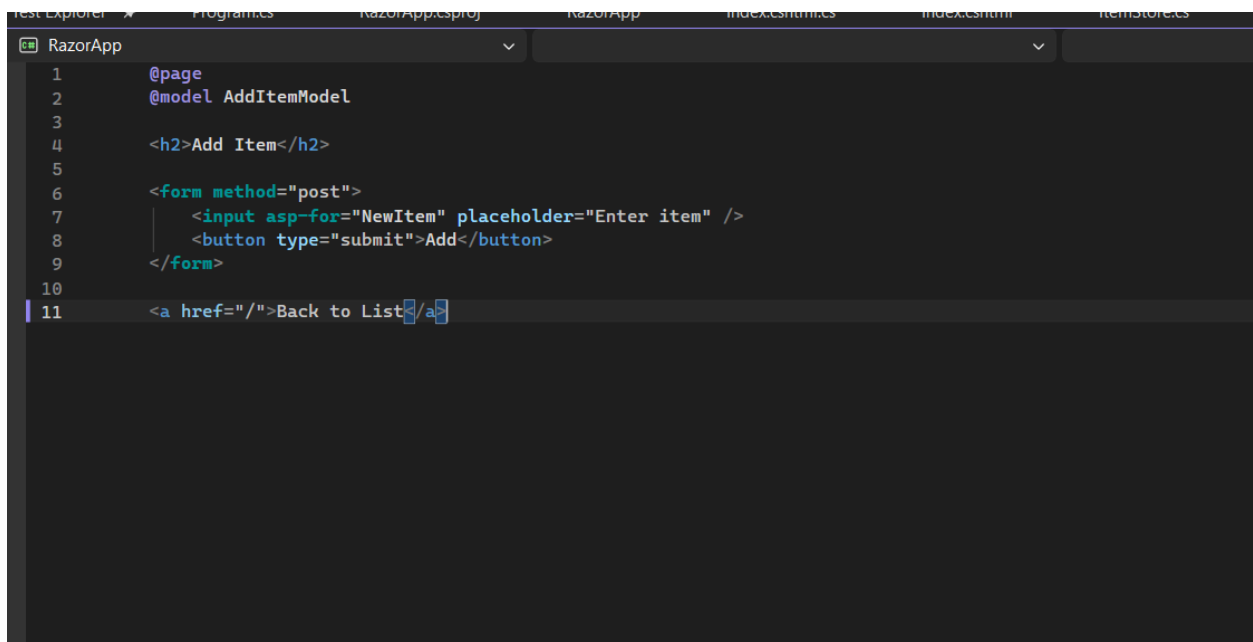
<form method="post">

    <input asp-for="NewItem" placeholder="Enter item" />

    <button type="submit">Add</button>

</form>

<a href="/">Back to List</a>
```

A screenshot of a code editor window. The top bar shows several tabs: 'Test Explorer', 'Program.cs', 'RazorApp.csproj', 'RazorApp', 'index.cshtml', 'index.cshtml', and 'ItemStore.cs'. The active tab is 'RazorApp'. The editor displays the following code:

```
1  @page
2  @model AddItemModel
3
4  <h2>Add Item</h2>
5
6  <form method="post">
7      <input asp-for="NewItem" placeholder="Enter item" />
8      <button type="submit">Add</button>
9  </form>
10
11 <a href="/">Back to List</a>
```

AddItem.cshtml.cs:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using RazorApp;

public class AddItemModel : PageModel
{
    [BindProperty]
    public string? NewItem { get; set; }

    public IActionResult OnPost()
    {
        if (!string.IsNullOrEmpty(NewItem))
        {
            ItemStore.Items.Add(NewItem);
        }

        return RedirectToPage("Index");
    }
}
```

```

1  using Microsoft.AspNetCore.Mvc;
2  using Microsoft.AspNetCore.Mvc.RazorPages;
3  using RazorApp;
4
5  public class AddItemModel : PageModel
6  {
7      [BindProperty]
8      public string? NewItem { get; set; }
9
10     public IActionResult OnPost()
11     {
12         if (!string.IsNullOrEmpty(NewItem))
13         {
14             ItemStore.Items.Add(NewItem);
15         }
16
17         return RedirectToPage("Index");
18     }
19 }

```

Output

